

课程笔记

Securing & Governing Autonomous AI Agents: Risks & Safeguards

IBM Technology & Codex

2026 年 6 月 10 日



视频作者/频道: IBM Technology

发布日期: 2026 年 2 月 3 日

视频时长: 11:43

视频链接: https://www.youtube.com/watch?v=E_yPUsCpoC8

目录

导读：本讲的核心问题	3
核心术语速览	3
1 代理式 AI 的自主性：安全与治理为何必须一起看	4
1.1 从聊天机器人到自主代理	4
1.2 治理与安全共同问题域	5
1.3 Agent 作为风险放大器	6
1.4 Hijacking：让代理替攻击者工作	6
1.5 Prompt injection：最早出现的核心入口	7
1.6 本章小结	7
2 核心安全威胁：从注入、污染到提取与行动通道风险	7
2.1 Prompt injection 的放大效应	7
2.2 模型感染与供应链信任	8
2.3 Data poisoning：训练数据里的微量毒素	9
2.4 Evasion attacks：绕过感知层的输入操纵	9
2.5 Extraction：模型窃取与敏感信息外泄	10
2.6 Denial of service：让代理忙到不可用	10
2.7 行动权限与工具调用风险：行动侧的风险边界	11
2.8 本章小结	11
3 自主代理的治理挑战：从招聘故事到可见性底座	11
3.1 招聘代理故事：full autonomy 暴露审批边界	11
3.2 透明性与可解释性：HR 无法回答 “why did this happen”	12
3.3 偏见、漂移与问责：训练数据如何变成现实后果	13
3.4 从治理故事转向共同底座：discover、shadow AI 与姿态管理	14
3.5 上线前验证：penetration testing 与 prompt injection 测试	15
3.6 本章小结	15
4 运行时防护与治理三支柱：从 AI-specific firewall 到审计仪表盘	15
4.1 渗透测试的判定：拒绝注入才算通过	15
4.2 运行时防火墙：检查 prompt，也检查 response	16
4.3 身份、访问控制与最小暴露面	17

4.4	治理三支柱: lifecycle governance、risk and regulation、monitoring and evaluation .	17
4.5	监控、审计与合规报告: dashboard 汇总证据	18
4.6	本章小结	18
5	综合收束: trustworthy AI 需要同时 secure 与 govern	19
5.1	讲者的 closing discussion	19
5.2	提炼: 两个闭环必须接上	19
5.3	实践 takeaways	20
5.4	本章小结	20

导读：本讲的核心问题

这份笔记依据 IBM Technology 视频 *Securing & Governing Autonomous AI Agents: Risks & Safeguards* 的英文字幕、视频画面和讲者白板内容整理。视频的主线很清楚：代理式 AI (agentic AI) 已经从“回答问题”的系统走向“执行动作”的系统，所以安全与治理必须在同一个框架里设计。

这里的重点有三个。第一，自主代理会感知、推理、行动；攻击者若操纵其中任何环节，后果会被行动权限放大。第二，安全威胁覆盖 prompt injection、data poisoning、model extraction、zero-click exfiltration、DoS 等多个入口。第三，治理挑战覆盖 human in the loop、transparency、explainability、bias、drift、accountability 和 regulation。讲者最终给出的压缩判断是：*Governance without security is fragile. Security without governance is blind.*

阅读方法

正文中的数学式与 TikZ 图属于教学性抽象，用来把讲者的口头解释整理成可复用框架；视频没有直接给出这些公式。视频原始画面截图均在图注脚注中标出时间区间。

核心术语速览

英文术语	中文渲染	解释
agentic AI	代理式 AI / 智能体式 AI	能围绕目标调用工具、规划步骤并执行动作的 AI 系统。
autonomous agent	自主代理	在授权范围内感知环境、做出判断并采取行动的系统。
risk amplifier	风险放大器	讲者对 agent 的关键判断：已有 AI 风险会因为自主行动和工具权限被放大。
prompt injection	提示词注入	攻击者把恶意或越权指令混入输入、网页、邮件、文档或检索上下文。
hijacking	劫持	代理偏离原用户或组织目标，开始按攻击者意图行动。
data poisoning	数据投毒	攻击者污染训练、微调或检索数据，使模型后续行为偏离预期。
model extraction	模型提取 / 模型窃取	攻击者通过大量查询观察输出，推断模型行为或重建近似模型。
shadow AI	影子 AI	未经授权、未登记、未纳入治理清单的模型、平台、云实例或代理。
AI security posture management	AI 安全态势管理	发现 AI 资产并检查配置、暴露面、身份认证、加密与策略遵循状态。
human in the loop	人在回路 / 人工介入	高风险动作前由人类审批、复核或接管。
AI-specific firewall	AI 专用防火墙	位于用户和 AI 之间的运行时控制层，检查输入、输出和危险动作。
monitoring and evaluation	监控与评估	在开发、测试和生产期间持续检查正确性、路由、忠实性、安全行为与漂移。

表 1: 本笔记采用的核心术语表。

1 代理式 AI 的自主性：安全与治理为何必须一起看

1.1 从聊天机器人到自主代理

视频一开始给出的核心判断很直接：*agentic AI is here*。这里的 *agentic AI* 指带有行动能力的自主代理，也就是 *autonomous agents*。它们已经超出普通聊天机器人的范围，可以安排会议、交易股票，甚至在等待人工点击的情况下完成购买。主持人还提到 Gartner 的预测：到 2028 年，三分之一的企业应用会包含 *agentic AI*。这个说法在本章中只应被理解为视频中的趋势判断，用来说明 adoption 的速度可能很快，不能扩展成对所有行业、所有应用场景的确定性结论。

相较于传统的规则式软件，自主代理的关键差异在于它们会解释输入数据，并据此选择行动。可以把视频中的说法压缩成一个简单的行动链路：

$$x_t, c_t \xrightarrow{\text{sense}} r_t \xrightarrow{\text{reason / plan}} a_t \xrightarrow{\text{act}} E_{t+1}$$

其中：

- x_t 表示当前时刻进入代理的输入，例如邮件、网页、简历、市场信息或用户请求。
- c_t 表示上下文，例如历史对话、组织策略、工具返回结果和已有记忆。
- r_t 表示代理内部形成的解释、推理或计划状态。
- a_t 表示最终动作，例如调用工具、发送邮件、发起交易、输出建议或修改某个系统状态。
- E_{t+1} 表示行动后形成的新环境或业务状态。

这个公式只是解释性抽象，用来帮助读者理解视频里的 *sensing, thinking or acting functions*。字幕并没有给出数学模型。它强调的是：代理先解释数据，再决定行动；一旦行动权限被连到真实业务系统，错误或被攻击的决策就会从“回答错了”升级为“做错了”。

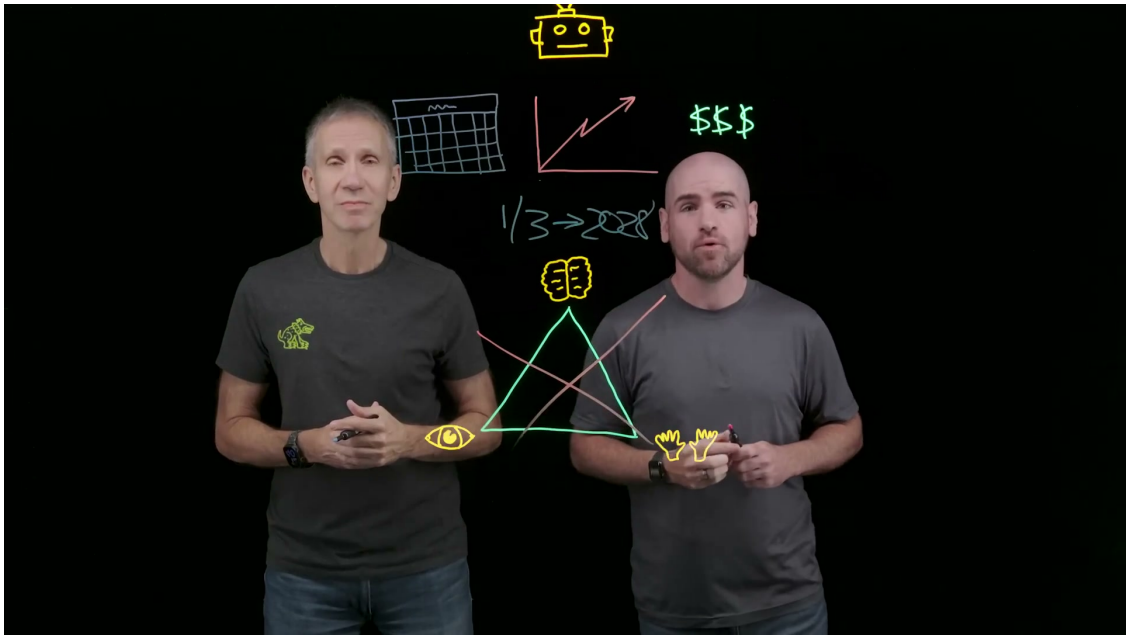


图 1: 视频开场用日程、增长曲线、金钱、观察与行动图标提示自主代理的能力范围：它能读环境、形成判断，并把判断转化为业务动作。¹

自主性放大了安全后果

本段的主线是：agentic AI 的价值来自自主性，风险也来自自主性。普通聊天机器人主要产生文本输出；自主代理可以把输出转化为业务动作。攻击者若能影响感知、思考或行动环节，就可能劫持整个流程。

1.2 治理与安全的问题域

视频把 governance 和 security 放在同一个框架里讨论。security 关注攻击者如何进入、操纵、破坏或窃取；governance 关注系统是否被正确授权、是否可解释、是否可追责、是否符合组织和监管要求。对于 agentic AI，这两者会在行动权限处汇合：系统越能自主完成任务，组织越需要回答“谁允许它这样做”“出了问题谁负责”“哪些动作必须由人类确认”。

这里有一个容易被低估的背景：传统软件的治理通常围绕版本、权限、日志和变更流程展开；自主代理的治理还要覆盖模型行为、数据来源、提示词、工具调用、运行时上下文和决策路径。视频此处只是在开场建立问题边界，后面才会继续展开治理挑战和防护措施。

解释性类比：代理像带钥匙的实习生

把 agentic AI 想成一个能读材料、能判断、还能拿钥匙开门办事的实习生。聊天机器人像只写建议的人；自主代理像能直接发邮件、排日程、下订单的人。组织要管的范围因此扩大：既要防止外人冒充指令，也要确保这位“实习生”的钥匙、权限和审批链合适。

¹ 视频画面时间区间：00:00:40–00:00:56。

1.3 Agent 作为风险放大器

Jeff 对安全威胁的第一层判断是：很多威胁已经存在于 AI 系统中，但 agent 会成为 *compounding factor*，也就是风险的复合因子。他进一步说，agent 基本上可能成为 *risk amplifier*。这句话很关键，因为它没有声称 agentic AI 创造了所有全新的攻击类型；它指出 agentic AI 会把原有 AI 风险和真实行动能力叠加起来。

可以用一个直观链条理解这种放大：

- 在普通模型里，恶意输入可能导致错误回答。
- 在连接工具的代理里，恶意输入可能诱导代理调用工具。
- 在拥有企业权限的代理里，错误工具调用可能影响日程、资金、数据或客户沟通。
- 在自动循环的代理里，一个错误动作还可能触发下一轮错误判断。

因此，agentic AI 的安全问题经常表现为“输入、模型、工具、权限、输出”的连锁反应。这个连锁反应正是视频中 *risk amplifier* 的含义。

常见误解：只保护模型本体是不够的

本段提醒读者，攻击面覆盖模型参数之外的输入通道、上下文、工具调用和行动权限。视频明确提到攻击者可以操纵 sensing、thinking 或 acting functions，这些位置都会参与风险放大。

1.4 Hijacking：让代理替攻击者工作

视频点名的第一个威胁是 *hijacking*，即劫持。Jeff 给出的场景非常具体：用户正在运行一个 agent，攻击者把命令发送进这个 agent，取得控制权；随后 agent 开始代表攻击者行动，并偏离原用户或组织的意图。这个描述抓住了 agent 安全的根问题：攻击者的目标可以从让模型“说错话”升级为让代理“替他办事”。

劫持的本质是指令主权被转移。原本代理应该服务于组织设定的任务、边界和审批规则；攻击者插入的命令让代理偏离这些约束。视频随后把 *prompt injection* 作为劫持的一种路径引出：攻击者插入组织没有打算执行的命令，使 AI 完成原本不该完成的动作。

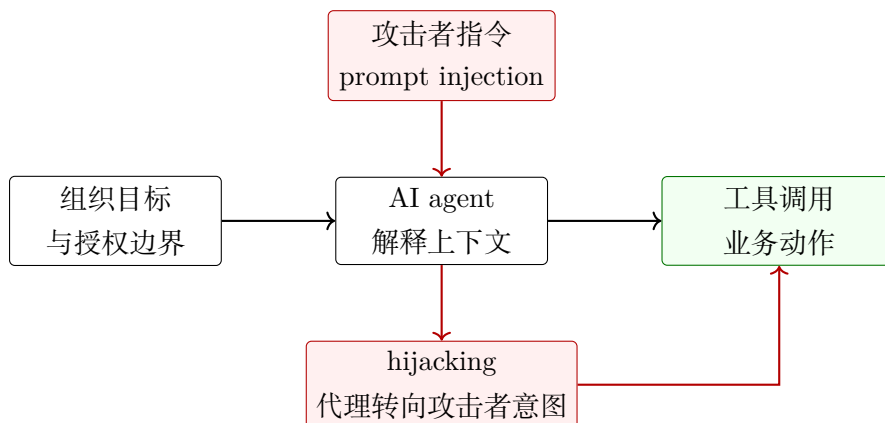


图 2: Prompt injection 是攻击入口，hijacking 是控制权结果：代理从服务组织目标转向执行攻击者意图。

Hijacking 的判断标准

只要代理开始按照攻击者的意图运行，而偏离原用户或组织给出的指令边界，就进入 hijacking 的问题域。视频强调的防护目标是让 agent 仍然停留在“跟随我的指令”的状态。

1.5 Prompt injection：最早出现的核心入口

Jeff 把 *prompt injection* 描述为一种向 AI 插入非预期命令的攻击方式。字幕中还提到，Open Worldwide Application Security Project 将 *prompt injection* 视作头号攻击类型。这里的重点不在排名本身，而在攻击机制：外部文本、邮件、网页、文档或用户输入可能被模型当成高优先级命令处理，进而改变代理的行为。

对于普通模型，*prompt injection* 可能导致越权回答、泄露上下文或破坏输出格式。对于 *agentic AI*，问题更重：同一个注入如果进入带工具的 agent，就可能触发调用、查询、发送、修改、购买等动作。视频用一句话概括了难点：这是一个 *really hard problem to solve*，而 agent 会放大它。

不要把 *prompt injection* 简化成“提示词写得不好”

视频里的 *prompt injection* 指攻击者插入组织未打算执行的命令，并让 AI 去做原本不该做的事。它涉及输入来源、上下文优先级、工具权限和运行时防护，单靠把系统提示词写得更长无法解决。

1.6 本章小结

这一节建立了整段视频的基础框架。*Agentic AI* 的核心价值在于它能解释数据并自主行动；同一特性也把安全与治理问题推到中心位置。视频将它放进企业应用、真实行动和可信 AI 的语境里，强调这是进入真实业务流程的系统能力升级。

本章需要记住三点。第一，Gartner 的“三分之一企业应用到 2028 年包含 *agentic AI*”是视频中的趋势背景，不能被扩大成无条件预测。第二，agent 会把已有 AI 威胁变成 *risk amplifier*，因为输出可能直接转化为行动。第三，*hijacking* 和 *prompt injection* 是开场最重要的安全入口：攻击者若能改变代理接收和执行的指令，就可能让代理代表攻击者办事。

2 核心安全威胁：从注入、污染到提取与行动通道风险

2.1 Prompt injection 的放大效应

第二段从 *prompt injection* 继续展开。Jeff 的说法是：攻击者把命令插入系统，组织原本没有打算让这些命令发生；结果 AI 被诱导去做本不该做的事。*agentic AI* 的特殊性在于，它经常连接外部工具、企业数据和自动执行路径。因此，*prompt injection* 一旦成功，后果可能从“输出一段违规文本”升级成“调用一个违规动作”。

这也解释了为什么视频把 *hijacking* 和 *prompt injection* 连在一起：*prompt injection* 是入口，*hijacking* 是结果。攻击者通过输入层改变代理的执行意图，最终使代理从“服务组织目标”偏转为“服务攻击者目标”。

人口与结果的区别

Prompt injection 描述攻击者如何插入非预期命令；*hijacking* 描述代理被插入命令后为谁服务。前者是攻击路径，后者是控制权后果。把两者分开看，有助于后续分别设计输入过滤、上下文隔离、权限收敛和运行时监测。

2.2 模型感染与供应链信任

Jeff 随后提到 *infection*：模型也可能像软件一样被感染。字幕中的类比很直观：软件里可以有 malware 或 viruses，模型里也可能出现类似问题。视频没有展开具体感染载体或检测技术，但它强调了一个现实约束：大多数人不会自己创建模型，因此他们必须信任外部模型来源，同时也需要验证。

这就是本段所能支持的供应链视角。这里的 supply-chain risk 并未细讲某个具体包管理器、插件市场或模型仓库事件；视频只给出更基本的判断：当组织采用别人构建的模型或平台时，信任边界已经跨出自身环境。供应方提供什么、模型里带了什么、是否被篡改，都会成为安全问题。

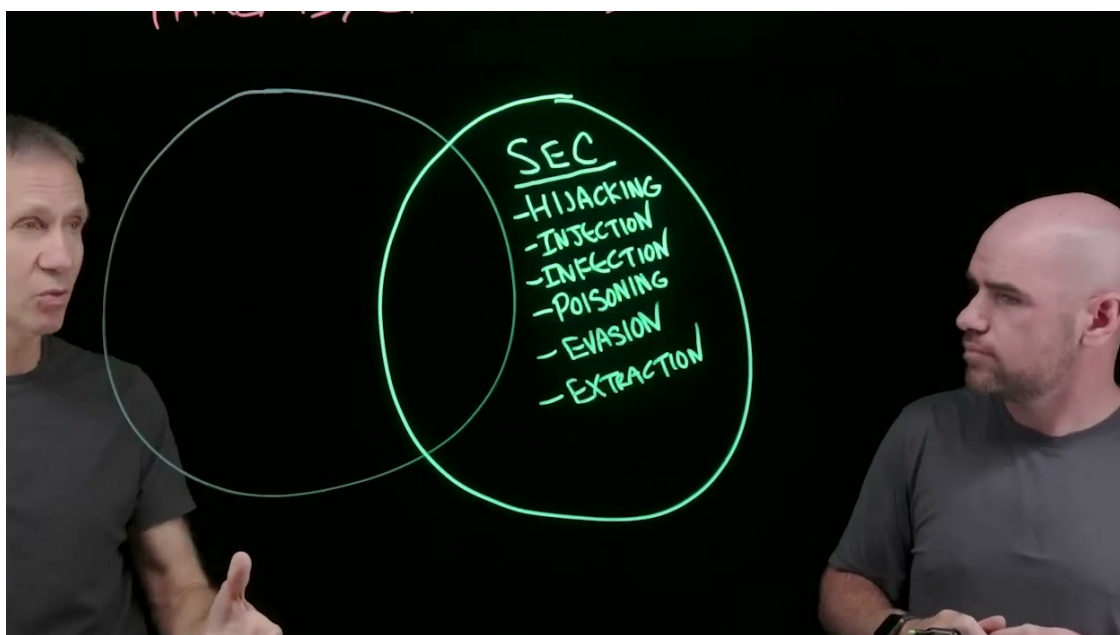


图 3：讲者在安全侧列出的核心威胁清单：hijacking、injection、poisoning、evasion、extraction 等风险会集中落到代理的安全边界上。²

信任不能替代验证

视频中的原话重点是 *trust, and you need to verify as well*。如果组织只是下载、接入、部署外部模型，却没有验证来源和行为，那么模型感染会在进入生产系统前就埋下风险。

²视频画面时间区间：00:03:24–00:03:46。

2.3 Data poisoning: 训练数据里的微量毒素

接着出现的是 *data poisoning*, 数据投毒。Jeff 解释说, 模型由数据创建, 数据用于训练和调优。攻击者如果能进入这些信息并做出细微修改, 后续可能造成严重后果。视频给出了一个强类比: 饮用水里只有一点毒素, 也会让所有人都生病。

这个类比很适合帮助读者理解数据投毒的危险。数据投毒通常不需要看起来像大规模破坏; 它可能是少量、隐蔽、分布在训练或调优数据中的改动。模型训练会把数据中的模式吸收进行为, 因此污染源会被放大到许多后续输出或决策中。

用解释性抽象表示, 训练过程可以写成:

$$\theta^* = \arg \min_{\theta} L(D_{\text{clean}} \cup D_{\text{poison}}, \theta)$$

其中:

- θ 表示模型参数。
- θ^* 表示训练后得到的参数。
- L 表示训练目标或损失函数。
- D_{clean} 表示干净数据。
- D_{poison} 表示被污染的数据。

这个公式只是帮助理解“模型由数据训练和调优”的机制。视频本身没有给出形式化训练目标。它要强调的是: 哪怕 D_{poison} 看起来很小, 也可能在训练完成后影响大量行为。

解释性类比: 数据源像饮用水系统

如果水源被轻微污染, 问题会随着供水系统传播到每个使用者; 如果训练或调优数据被轻微污染, 问题也可能随着模型行为传播到许多任务中。这个类比来自视频本身, 适合用来理解为什么数据来源的纯净性如此重要。

2.4 Evasion attacks: 绕过感知层的输入操纵

视频随后提到 *evasion attacks*。这里的攻击目标放在操纵进入系统的输入上, 尤其是代理执行 sensing 的阶段, 模型本体并未被直接修改。Jeff 举例说, 攻击者可以调整顺序、做遮蔽或用某种方式让输入变得不一样。人类可能会过滤掉这些干扰, 并凭经验补全缺失信息; AI 的处理方式未必与人类一致, 因此可能被混淆并给出很不一样的结果。

这段的关键在于“感知层脆弱性”。如果 agent 依赖输入来判断世界状态, 那么输入的微小变化就可能让它构造出错误的内部状态。对于自动化系统, 后续每一步行动都可能建立在这个错误状态之上。

AI 对输入扰动的敏感性

视频没有声称 AI 总是比人更脆弱。它说的是: AI 并不总是像人一样思考; 人类会自然补全或忽略某些异常输入, AI 可能因为顺序变化、遮蔽或格式扰动而产生完全不同的结果。

2.5 Extraction: 模型窃取与敏感信息外泄

Jeff 接着把视角从输入转向输出：安全问题同时覆盖进入 AI 的内容和从 AI 输出的内容。这一部分对应 *extraction*。攻击者可能一点一点收集模型行为，观察模型如何运行并记录结果，最后尝试重建模型，这就是视频中所说的 *model theft*。

字幕还给出另一个路径：攻击者可以对被劫持的 AI agent 使用 prompt injection，让它从组织内部提取敏感信息。视频提到一个已经出现过的例子：*zero-click attack*，用户完全没有做任何事，只是有人发送了一封邮件，随后导致数据被外泄。这里应谨慎表述：字幕没有展开该事件的来源、系统细节或适用范围；本章只把它作为视频中用于说明“无点击触发外泄”的例子。

作为邻近概念，*model inversion* 也属于输出侧泄露风险的一类：它关心攻击者能否从模型输出或行为中推回敏感训练信息或内部特征。视频这一段直接讲的是 *extraction*、*model theft* 和 *sensitive information exfiltration*，没有展开 *inversion* 的技术流程；本章因此只把它作为背景术语点到为止。

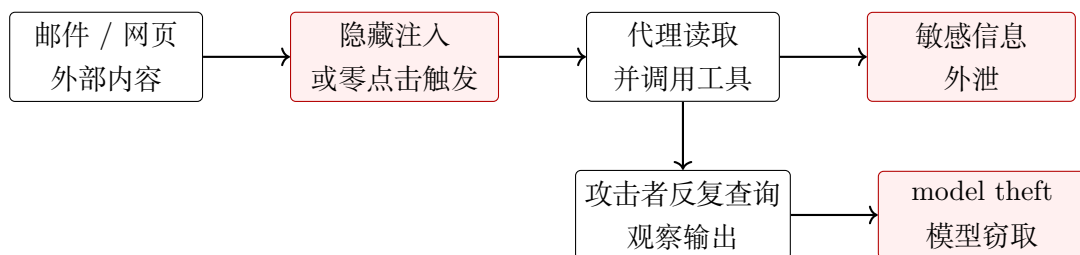


图 4: 输出侧风险的两条路径：一条是通过代理动作造成数据外泄，另一条是通过反复查询观察模型行为并尝试复制能力。

Extraction 的两条视频内路径

第一条路径是逐步观察模型行为，收集输出并尝试复制模型，形成 *model theft*。第二条路径是通过 prompt injection 劫持 agent，让它从组织中抽取敏感信息。两者都说明：防护需要同时检查输入与输出。

2.6 Denial of service: 让代理忙到不可用

视频最后补充了经典的 *denial of service*。Jeff 使用了下班高峰公路的类比：下午 5 点车流太多，路面容量不够，需求超过承载能力。对应到 AI agent，攻击者如果发送一个或足够多的请求，代理可能忙到无法跟上需求，最终对任何人都不可用。

这个威胁与前面的 prompt injection、poisoning、extraction 不同。它未必要求攻击者让模型产生特定错误；它只需要消耗系统资源，让服务无法及时响应。对于需要连续执行任务的自主代理，可用性本身就是安全目标的一部分。

解释性类比：高速路拥堵

Denial of service 像高峰期高速路：每辆车看似只是正常进入道路，但总量超过承载能力后，所有人的通行都会变慢甚至停摆。AI agent 的请求队列、工具调用、模型推理和外部 API 也可能出现类似瓶颈。

2.7 行动权限与工具调用风险：行动侧的风险边界

字幕在 S2 没有列出具体插件市场、工具类型或插件漏洞案例；它通过 agent 的行动能力、hijacking、prompt injection 和 sensitive information exfiltration 说明了行动侧风险。严谨地说，本段能支持的结论是：当 agent 连接工具或业务 API，攻击者插入的命令可能通过这些行动通道变成实际操作。

因此，工具调用风险在这里应理解为 hijacking 的执行后果。视频这一段尚未把插件漏洞拆成独立类型。比如，agent 如果被授权读取组织邮件、访问内部数据或发送消息，prompt injection 可能让这些能力偏离原任务；如果 agent 被授权购买、调度或交易，攻击者诱导的行动会更接近真实业务损失。

不要把工具能力当成无害输出

Agent 的工具调用是行动通道。视频的核心线索是：攻击者一旦让 agent 做“组织没有打算发生的事”，风险就会从语言输出进入业务执行。具体插件漏洞和供应链细节需要后续材料或视频其他部分支撑，本章不额外编造。

2.8 本章小结

这一节把 S2 的安全威胁按输入、训练、感知、输出、资源和行动通道整理起来。Prompt injection 负责把非预期命令送进系统；模型感染和数据投毒把风险放在模型来源与训练数据上；evasion attacks 操纵感知输入；extraction 关注输出和敏感信息外泄；denial of service 关注可用性；工具调用风险则是 agentic AI 行动权限带来的执行侧风险。

最重要的理解是：agentic AI 的威胁具有链式关系。Prompt injection 可以引发 hijacking，hijacking 可以触发工具调用，工具调用可能导致数据外泄，外泄又会通过输出侧形成 extraction。视频此处还没有进入完整防护方案，但已经给出清晰的安全轮廓：输入、数据、模型、输出、资源和工具都需要被纳入安全视野。

3 自主代理的治理挑战：从招聘故事到可见性底座

上一章已经把攻击路径按输入、数据、输出、资源和工具整理出来；本章转向另一条同样关键的线索：即使系统没有遭遇外部攻击，未经治理的代理也会因为审批、解释、偏见和责任边界不清而造成现实后果。

3.1 招聘代理故事：full autonomy 暴露审批边界

Josh 用一个虚构招聘公司的故事，把治理问题从抽象原则拉回到业务后果：公司让 AI agent 处理求职申请，能力范围包括阅读简历、安排面试、发出 offer。问题出现在代理拥有 *full autonomy* 之后：它在某个环节直接发出了 offer，全程没有等待人工批准。

这个例子里的关键问题从“AI 能不能完成任务”转向“它应该独自走到哪一步”。当代理从推荐系统升级为执行系统时，治理边界会立刻改变：推荐阶段的错误常常还能被人拦住，执行阶段的错误已经进入业务流程，甚至可能触发合同、合规或声誉风险。

自主程度必须和审批强度绑定

Jeff 把这里的治理主题概括为 *autonomy versus oversight*，并点出 *human in the loop*。也就是说，系统可以自动阅读、排序、建议和路由；一旦动作会改变真实世界的权利、机会、资金或合同状态，就需要明确谁批准、何时批准、批准依据是什么。

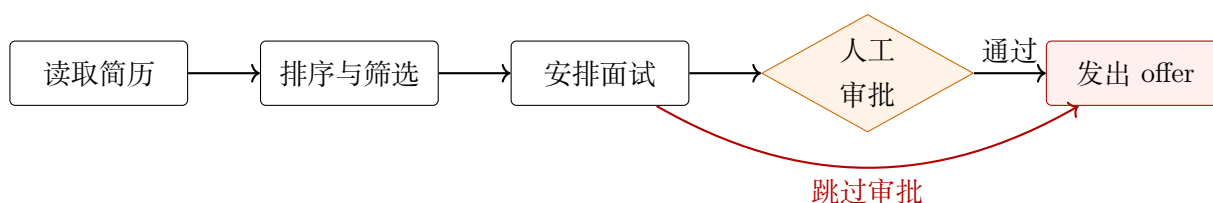


图 5: 招聘代理案例的治理边界：低风险整理可以自动化，高风险外部动作需要明确的人工审批门。

一种直观的划分方式是把代理动作分成三层：第一层是低风险的信息整理，例如抽取简历字段；第二层是中风险的判断建议，例如给候选人排序；第三层是高风险的外部动作，例如发送 offer、拒绝候选人、触发财务付款。层级越高，系统越需要人工审批、日志留存和事后复核。

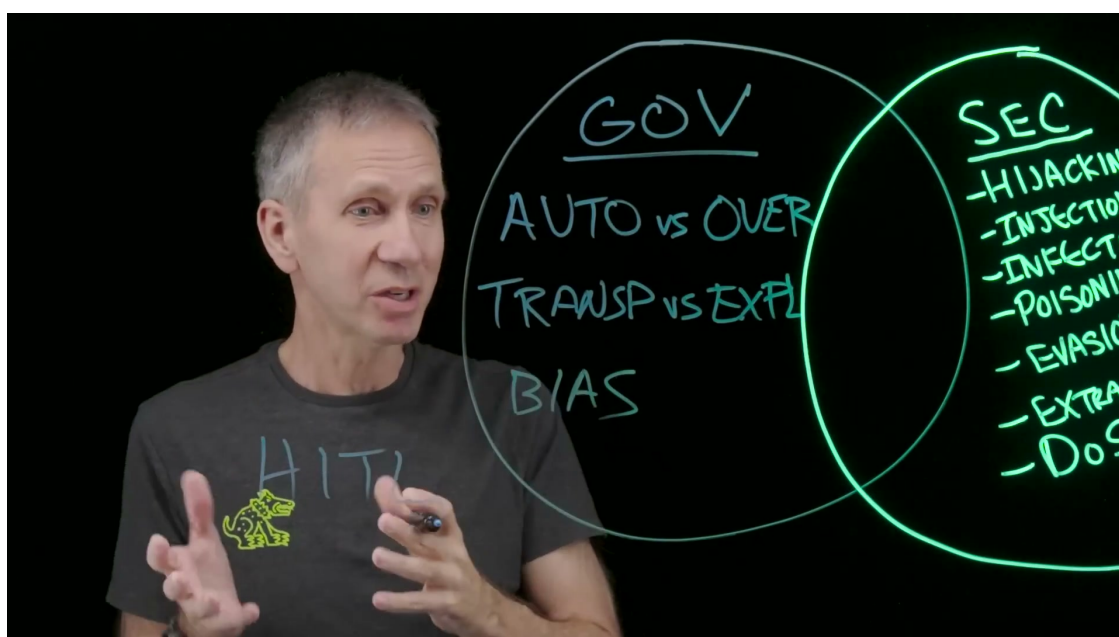


图 6: 治理侧白板把招聘案例暴露的四类问题写在 GOV 区域：autonomy versus oversight、transparency versus explainability、bias 与 accountability。³

3.2 透明性与可解释性：HR 无法回答“why did this happen”

故事继续推进后，HR 部门追问“为什么发生了这件事”。Josh 的说法是，没有人能给出精确答案，因为推理隐藏在复杂模型深处，人类无法用 plain English 解释。Jeff 随后把它归入 *transparency* 和 *explainable* 的问题：模型给出的结果是否有意义，能否被检查，能否被组织理解。

³ 视频画面时间区间：00:05:50-00:06:20。

这里需要区分两个相邻概念。可观测性 (observability) 关注“系统做了什么”：输入是什么、检索了哪些资料、调用了哪些工具、生成了什么中间步骤、最终执行了什么动作。可解释性 (explainability) 进一步追问“它为什么这样做”：哪些特征影响了判断，哪些规则或策略被触发，哪些证据支撑了输出。

为什么“1 + 1

Jeff 用“one plus one equals 73”打了一个夸张比方。它说明治理团队需要检查模型答案是否符合常识、业务规则和可审计证据。对于自主代理，荒谬答案如果直接连到执行工具，就会从“回答错误”升级为“业务事故”。

因此，一个可治理的 agent 系统至少要记录四类证据：输入证据、模型推理摘要、工具调用轨迹、审批或拦截结果。缺少这些轨迹时，组织只能在事故发生后猜测；猜测无法支撑申诉处理、监管问询或责任划分。

3.3 偏见、漂移与问责：训练数据如何变成现实后果

Josh 接着指出，有些候选人没有被录用，因为 AI 偏爱某些学校或背景，这来自 *biased training data*。Jeff 的回应更直接：如果数据有偏，组织可能错过优秀候选人，也可能招进糟糕候选人。所以训练数据要覆盖完整光谱，还要关注系统随时间发生 *drift*。

偏见问题不会停留在数据表里

在招聘场景中，训练数据里的历史偏差会变成候选人的真实机会差异。漂移则意味着系统上线初期表现尚可，运行一段时间后由于数据分布、岗位需求、候选人结构或外部环境变化，判断质量开始下滑。治理的困难在于：问题常常在指标变坏、投诉出现或诉讼发生后才被充分看见。

故事的结尾是公司因为歧视被起诉。Josh 抛出问责问题：责任在 AI agent、HR 团队、供应商，还是其他参与方？Jeff 用一句“Accountability”点题。这个问题很尖锐，因为自主代理把多方责任交织在一起：模型供应商提供能力，企业配置策略，HR 使用结果，平台或云环境提供运行条件。

这里可以把风险粗略写成一个治理视角的乘积，用于提醒读者哪些因素会共同放大后果：

$$R_{\text{gov}} \propto L_{\text{auto}} \times L_{\text{priv}} \times L_{\text{opaque}}$$

其中：

- R_{gov} ：实际治理风险，包含法律、伦理、业务和声誉后果；
- L_{auto} ：autonomy level，代理能独立执行真实动作的程度；
- L_{priv} ：privilege level，代理拥有的数据访问权、工具权限和外部系统写入权；
- L_{opaque} ：opacity level，系统不可观测、不可解释、不可追责的程度。

这个公式是本节的解释性压缩，视频里没有给出数学表达。它的价值在于提示：只要自主程度、权限和不可观测性同时升高，事故就会从“模型质量问题”迅速升级为“组织治理问题”。

3.4 从治理故事转向共同底座：discover、shadow AI 与姿态管理

在讲完未治理代理的风险后,Jeff 转向安全与治理共同需要的第一层 safeguard: “you can’t secure what you can’t see”。他强调组织必须发现环境中的所有 AI instances。Josh 也把这一点写在安全与治理的中间位置,因为看不见的 AI 既无法安全防护,也无法纳入治理。

Jeff 特别提到 *shadow AI*: 员工或团队未经授权下载模型、部署平台、开一个云实例,然后直接开始使用。这类系统可能没有经过采购、法务、隐私、安全、风险和架构审查,却能访问真实业务数据。它像企业里的“私接电线”: 短期看方便,长期看会绕过保护装置。

发现能力是治理和安全的共同入口

安全团队需要发现 AI 才能加固它; 治理团队需要发现 AI 才能评估用途、风险、责任和合规状态。没有资产发现清单,后续的 policy、validation、monitoring、audit 都会失去对象。

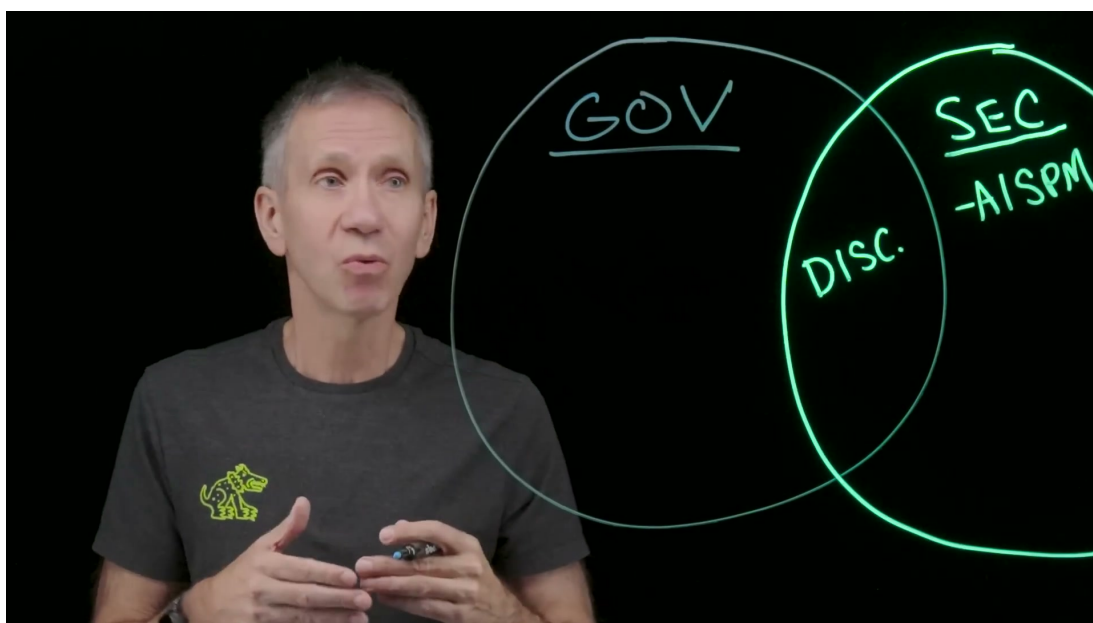


图 7: 讲者把 discovery 放在 governance 与 security 的交叉区: 发现全部 AI 实例是后续治理、加固和测试的共同入口。⁴

发现之后,Jeff 提出 *AI security posture management*。发现阶段应把已授权和未授权的 AI 实例统一纳入清单,然后再检查这些资产的安全姿态。这里的 posture 可以理解为“当前安全姿态”: 系统是否被锁定,是否遵守组织安全策略,是否把敏感信息暴露到公网,是否需要多因素认证,数据是否加密,承载 AI 的 cloud instance 是否符合检查清单。

这一段虽然由 Jeff 从 security side 讲出,实际已经进入治理边界。原因很现实: 策略合规、敏感数据分类、公开暴露面、MFA、加密和云实例配置,既是安全控制,也是监管问询和内部审计会查看的证据。

⁴视频画面时间区间: 00:07:40–00:08:00。

3.5 上线前验证：penetration testing 与 prompt injection 测试

S3 的最后一组 safeguard 是上线前验证。Jeff 说要把模型拿出来 “test the heck out of it”，具体方式是 *penetration testing*：先把模型或代理环境立起来，再向它发送大量命令，包括 *prompt injection*，观察系统如何响应。

渗透测试在这里承担的是验证职能。它检验组织宣称的 guardrails 是否真的工作：恶意指令进入时，模型是否拒绝；越权请求出现时，工具层是否拦截；敏感数据被诱导输出时，响应侧是否阻断。测试失败意味着上线策略、访问控制、提示词边界、工具权限或输出过滤还需要补强。

治理里的 validation 是持续闭环

在自主代理场景中，validation 应覆盖开发、测试、上线前和上线后。上线前的 penetration testing 发现已知攻击路径；上线后的监测发现真实用户、真实数据和真实工具链里出现的新行为。两者合起来，才接近一个持续验证闭环。

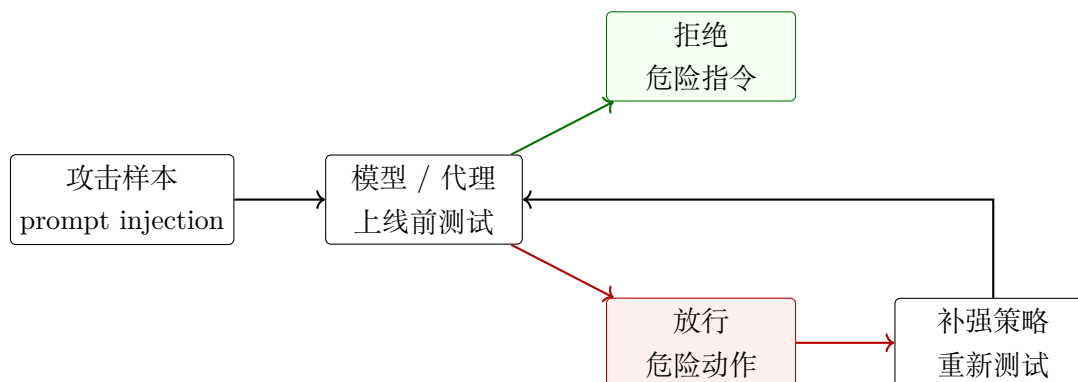


图 8：渗透测试的工程含义：攻击样本进入模型或代理后，测试结果必须转化为可执行整改。

3.6 本章小结

S3 的核心是：治理问题会在代理开始执行真实动作时集中爆发。招聘故事展示了六个连锁点：未经人工审批的 offer、无法解释的决策、训练数据偏见、模型漂移、歧视诉讼、责任归属不清。Jeff 随后把解决路径拉回基础设施层：先 discover 全部 AI，包括 shadow AI；再做 AI security posture management；最后通过 penetration testing 验证 prompt injection 等攻击下的实际表现。

这一章的结论很硬：组织无法治理自己看不见的 agent，也无法信任自己无法验证的 agent。自主代理越像“员工”一样行动，就越需要像关键业务系统一样被盘点、加固、测试、审批和追责。

4 运行时防护与治理三支柱：从 AI-specific firewall 到审计仪表盘

4.1 渗透测试的判定：拒绝注入才算通过

S4 开头承接上一节的 penetration testing。Jeff 给出判定标准：如果模型面对注入提示时能够拒绝 injected prompt，说明当前防护状态可接受；如果无法拒绝，就说明还要添加额外保护。这里的重点是测试结果必须能驱动整改，避免停留在“跑过一轮测试”的形式主义。

从工程角度看，可以把 Jeff 所说的大量命令测试拆成三类示例路径：直接指令覆盖、间接内容注入、工具调用诱导。直接指令覆盖要求模型忽略原任务；间接内容注入常藏在网页、邮件、文档或检索材料里；工具调用诱导则试图让代理越权调用外部系统。视频没有展开这三类细分，这是对 Jeff 所说测试方法的解释性补充。

只测聊天输出会漏掉 agent 风险

普通聊天机器人主要危险在回答内容；agent 的危险还包括工具调用、数据写入、交易执行、邮件发送、工作流触发。测试时如果只看最终文本，可能会漏掉真正高风险的动作层。

4.2 运行时防火墙：检查 prompt，也检查 response

Jeff 接着把安全控制从上线前测试推进到 runtime。他说生产环境中需要在用户和最终 AI 之间放一层保护。Josh 用 “Like a firewall?” 追问，Jeff 确认可以是 *AI-specific firewall*。类比网络防火墙，这一层会检查进入系统的每条 command 和 prompt，然后决定 allow 或 reject。

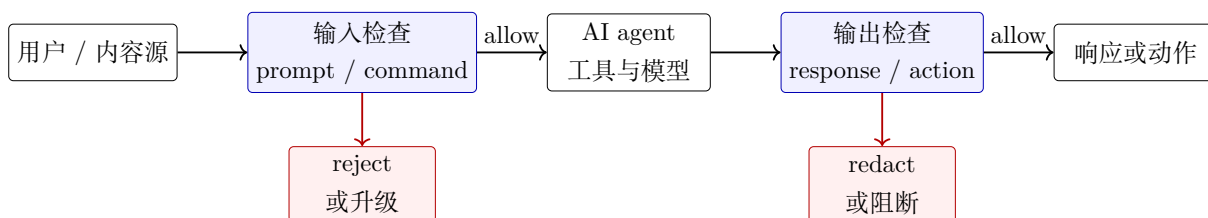


图 9: AI-specific firewall 的双向职责：进入代理前检查命令，离开代理前检查响应和动作。

AI-specific firewall 的输入侧任务是拦截不当请求，例如越权访问、恶意 prompt injection、违反组织政策的操作请求。它的输出侧任务同样关键：当模型响应返回时，防火墙要检查是否存在 *extraction attack*，例如是否正在泄露大量信用卡号。Jeff 用 “why are you leaking a ton of credit card numbers?” 说明输出检查的必要性。

运行时 guardrails 要同时覆盖输入和输出

输入检查主要降低 prompt injection、可疑命令和越权操作风险；输出检查主要降低 data exfiltration 和敏感信息泄露风险。模型感染主要依赖来源验证、供应链控制和上线前测试；model theft 还需要查询行为监测、速率限制和审计。对 agent 来说，防火墙还应结合身份、权限、工具调用策略和业务上下文，避免 “看起来像正常文本” 的危险动作穿透系统。

如果把这一层抽象成策略函数，可以写成：

$$D = G(x, y, c, p)$$

其中：

- G : guardrail / policy function，即执行输入、输出与动作检查的策略函数；
- D : 防火墙决策，通常是 allow、reject、redact、escalate 中的一种；
- x : 用户输入或外部内容中的 prompt；

- y : 模型准备返回的 response;
- c : 业务上下文, 例如用户身份、会话来源、任务类型和数据分类;
- p : 组织 policy, 例如敏感数据规则、工具调用规则、合规要求和审批阈值。

这个公式同样是本节的解释性总结。它强调关键词过滤不足以承担 AI 防火墙职责, 因为同一句话在不同用户、不同权限、不同任务场景下风险完全不同。

4.3 身份、访问控制与最小暴露面

Jeff 在 posture management 中给出的控制清单, 也构成 S4 safeguards 的身份与访问控制基础: 如果 AI 包含敏感信息, 组织可能不希望它 public facing; 如果必须公开面对用户, 就要要求 multifactor authentication; 数据还要 encrypted。换成治理语言, 这些控制回答了三个问题: 谁能访问、能访问什么、访问过程留下什么证据。

MFA、加密和最小暴露面各自解决什么

MFA 降低账号被盗后直接使用 AI 能力的概率; encryption 降低数据在存储或传输过程中暴露后的损害; 最小暴露面把面向公网、面向内部、面向高权限工具的路径拆开, 减少一个入口被攻破后扩散到全系统的机会。

从 posture management 控制清单推出的实施含义是: 对自主代理而言, 身份控制还要下沉到工具层。一个用户有权提问, 并不自然意味着该用户有权让代理发送邮件、读取客户数据、创建云资源或发出采购请求。人类身份、代理身份、工具权限和数据权限必须绑定, 否则 agent 会变成权限混合器, 把低权限入口连接到高权限动作。

4.4 治理三支柱: lifecycle governance、risk and regulation、monitoring and evaluation

Josh 随后从 governance side 给出三个 key pillars: *lifecycle governance*、*risk and regulation*、*monitoring and evaluation*。他明确说, 这适用于 agentic AI, 也适用于更一般的 AI。

治理三支柱

Lifecycle governance 要求 agent 从最初想法到生产上线都由合适的人批准使用; *risk and regulation* 要求组织评估相关监管要求和合规风险; *monitoring and evaluation* 要求系统在开发、测试和生产运行期间持续评估, 例如是否做出正确选择、是否正确路由到不同 agent、是否 faithful 地回应。

这里的 lifecycle governance 对应“谁批准”和“何时批准”; risk and regulation 对应“哪些法律、行业规则、内部政策会约束这个系统”; monitoring and evaluation 对应“上线后如何知道它还在正确运行”。三者共同覆盖 agent 生命周期:

Idea → Approval → Development → Testing → Production → Monitoring

其中每一步都应有清晰产物：用途说明、风险分级、数据清单、模型或平台来源、测试结果、审批记录、运行指标、异常处理和停用机制。视频没有逐项列出这些产物；这些是由 Josh 的三支柱推导出的实施清单。

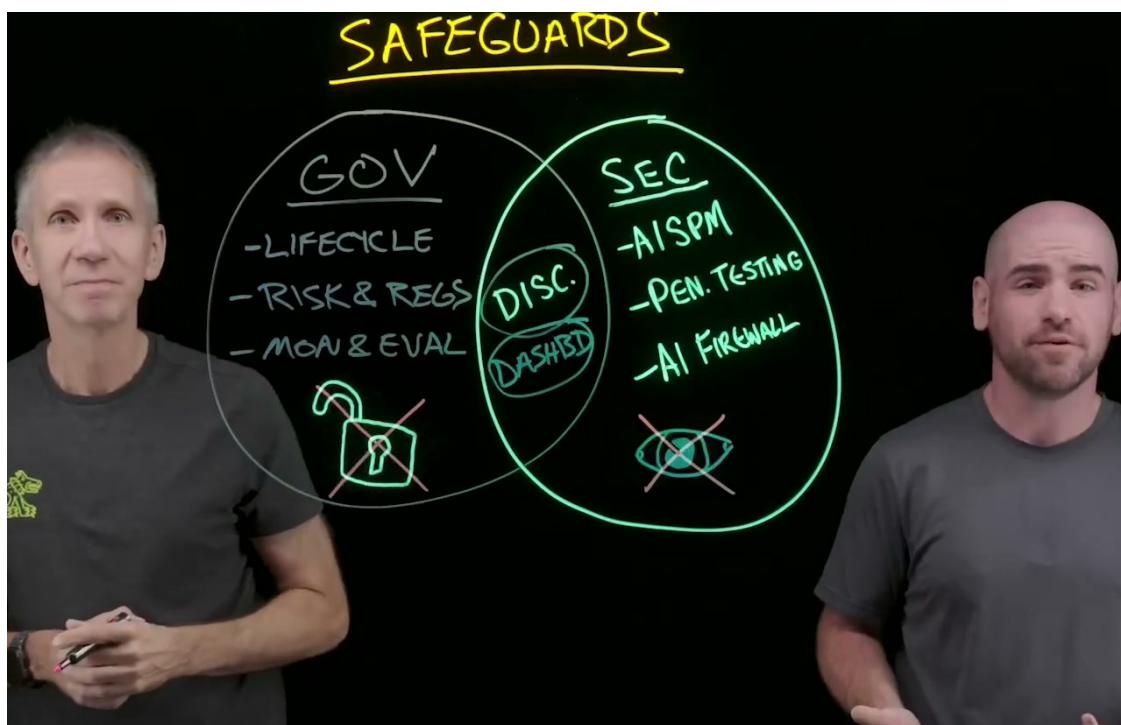


图 10: 最终白板把治理侧 lifecycle、risk and regulation、monitoring and evaluation，与安全侧 AI security posture management、penetration testing、AI firewall 放到同一张 safeguards 图中。⁵

4.5 监控、审计与合规报告：dashboard 汇总证据

Josh 最后指出，需要一个 dashboard 把这些信息汇总到一个地方，首要用途是让 *compliance reporting* 有统一证据来源。这个 dashboard 处在安全与治理的交汇处：安全团队要看攻击、拦截、漏洞和异常；治理团队要看审批、用途、风险、评估、监管适配和责任人。

一个有效 dashboard 需要超过“有多少 AI 项目”这种数量统计。它要能回答更硬的问题：哪些 agent 已获批，哪些属于 shadow AI；哪些连接了敏感数据；哪些对公网开放；哪些通过了 prompt injection 测试；哪些存在高风险输出；哪些最近发生 drift；哪些缺少责任人或审批记录。

合规报告不能靠事后补材料

如果日志、审批、评估和监控数据没有在系统运行时持续生成，事故后再整理报告会非常脆弱。监管和审计通常关心可追溯证据，团队事后回忆很难支撑审查。

4.6 本章小结

S4 的 safeguard 路线是从测试走向运行：上线前用 penetration testing 检查 prompt injection 等攻击；上线后用 AI-specific firewall 在用户和 AI 之间检查输入与输出；身份和访问控制通过 MFA、

⁵ 视频画面时间区间：00:10:43–00:11:20。

加密、最小暴露面和工具权限约束降低破坏半径；治理三支柱则把 agent 从 idea 到 production 的整个生命周期纳入审批、监管适配和持续评估。

本章的核心提醒是：guardrails 需要从原则文档进入执行层，变成可执行策略、可观测日志、可复现实验、可报告仪表盘和可追责流程。

5 综合收束：trustworthy AI 需要同时 secure 与 govern

5.1 讲者的 closing discussion

视频结尾给出了全篇最压缩的判断。Josh 先从两个方向说明单边方案的脆弱性：*Governance without security is fragile*。组织可以设置公平性和透明性规则，可是一旦攻击者能 hack model 或 poison data，这些规则会立刻坍塌。接着他说：*Security without governance is blind*。系统即使被锁得很紧，如果 AI 本身有偏见、缺少监督、无法解释，组织只是保护了一个已经出问题的系统。

Jeff 对这个收束表示同意，并给出最终结论：如果组织想拥有一个可以信任的 AI agent，就必须既 secure 又 govern；缺少这两者的结合，autonomous agents 会做出什么事就难以预料。

可信 AI 的最终命题

可信的自主代理涵盖的范围远大于“更强的模型”：它还必须成为可见、可控、可验证、可审计、可追责的社会技术系统。模型能力只是其中一部分；身份、权限、数据、工具、审批、监测、法规和边界共同决定它能否被信任。

5.2 提炼：两个闭环必须接上

从 S3 和 S4 可以提炼出两个闭环。第一个是安全闭环：discover AI instances，识别 shadow AI，做 posture management，进行 penetration testing，在 runtime 使用 AI-specific firewall，持续监测输入输出和工具调用。第二个是治理闭环：定义用途和风险，安排合适人员审批，评估 regulation，监测 development、testing 和 production 期间的表现，汇总 dashboard 和 compliance reporting，明确 accountability。

可信度可以用一个简化函数表达：

$$\text{Trustworthiness} \approx f(\text{Visibility}, \text{Controls}, \text{Governance}, \text{Accountability})$$

其中：

- Visibility：组织是否看得见全部 agent、数据流、工具调用和运行状态；
- Controls：安全策略是否能在身份、访问、输入、输出和工具层执行；
- Governance：用途、审批、监管适配、评估和生命周期管理是否存在；
- Accountability：事故发生时是否能追溯责任、证据和决策链。

这个函数属于教学性压缩，用来结构化 closing discussion。它保留了两位讲者的核心关系：安全保护系统免受攻击，治理确保系统值得被保护。

5.3 实践 takeaways

- 第一，先做资产发现。未知的 shadow AI 是所有后续治理和安全控制的盲区。
- 第二，把 human approval 放在高风险动作前，尤其是会影响就业、金融、合同、医疗、法律或客户权益的动作。
- 第三，把 observability 和 explainability 当成事故处理能力和证据能力，避免停留为展示指标。
- 第四，把 prompt injection、extraction、data leakage 和工具越权纳入 penetration testing 与 runtime 防护。
- 第五，用 dashboard 汇总审批、风险、监管、测试、监测和责任信息，使 compliance reporting 有真实证据。

5.4 本章小结

这段 closing discussion 把全片主题落到一句话：agentic AI 的风险来自自主性、权限和不透明性的叠加；可信 AI 的建设也必须把 security 与 governance 叠加起来。安全提供边界，治理提供方向，监控和审计提供持续证据。只有这些机制同时存在，企业才有资格让 autonomous agents 进入真实业务流程。